Concept Study Guide

Project: Auditable AI Agents
Purpose: every concept you need to understand and defend this project, in a sensible study order. Each entry has a definition in the sense this project uses it, where it appears in the code, links to read, and a question to test yourself.

About the links. Every link below was opened and checked on 2026-10-30. Pages change, so if one moves, search for its title. Wikipedia pages are good for definitions; the official documentation pages are the ones to cite or rely on for details.

Suggested pace: about 4 weeks, one block per week.

Week	Blocks	Goal
1	A. Cryptography	Explain hashes, hash chains and Merkle proofs without notes
2	B. Blockchain and contracts	Explain what the contract stores and why only roots go on-chain
3	C. Agents and models, D. Python and testing	Follow the code in src/ and run every test
4	E. Security, F. Research writing	Defend the threat model; start verifying citations

---

A. Cryptography foundations

1. Cryptographic hash function

Definition. A function that turns any input into a fixed-size fingerprint, such that the same input always gives the same output, a small change in input changes the output completely, and nobody can practically work backwards from the output or find two inputs with the same output.
In this project. Every log entry's fingerprint, every Merkle node.
Read. Cryptographic hash function (Wikipedia)
Check yourself. Why does changing one character of a log entry change its whole hash, and why is that useful?

2. SHA-256

Definition. A standard hash function that produces 32 bytes (64 hex characters). Defined in the US government standard FIPS 180-4.
In this project. The only hash used, in Python (`hashlib`) and in Solidity (`sha256`), so both sides agree. Code: `src/core/hashing.py`.
Read. FIPS 180-4, Secure Hash Standard (NIST) (the standard itself; skim), Python hashlib documentation
Check yourself. Why not use Python's `sha3_256` while the contract uses Ethereum's native hash?

3. Canonical serialisation (canonical JSON)

Definition. Writing structured data as text in exactly one agreed way so identical data always gives identical bytes (sorted keys, no spaces, UTF-8).
In this project. Entries are hashed over canonical JSON so a different machine recomputes the same hash. Code: `canonical_json`.
Read. Python json module (see `sort_keys` and `separators`)
Check yourself. What would go wrong if two programs wrote the same entry with keys in a different order?

4. Hash chain (as used in logs)

Definition. A sequence where each record includes the hash of the previous record, so changing, removing, inserting or reordering any record breaks every link after it.
In this project. `prev_hash` and `entry_hash` in each log entry. Code: `src/domain/chain.py`.
Read. The idea is explained in the project's own technologies.md section 3 and in the Crosby and Wallach paper (item 8). Note: the Wikipedia page Hash chain is about a different use of the same name (one-time passwords and signatures), so do not confuse the two.
Check yourself. Why can an attacker who recomputes all later hashes defeat a hash chain on its own?

5. Merkle tree

Definition. A binary tree of hashes: leaves are hashes of data blocks, each parent is the hash of its two children, and the top is a single root that commits to everything below.
In this project. One root per batch of entries. Code: `src/domain/merkle.py`.
Read. Merkle tree (Wikipedia)
Check yourself. If you change entry 3 of 8, which hashes change?

6. Merkle inclusion proof

Definition. The list of sibling hashes along the path from one leaf to the root. A verifier who knows only the root can recompute upward and confirm the leaf belongs, without seeing other entries.
In this project. `merkle_proof`, `verify_proof`, and `verifyInclusion` in the contract.
Read. The Wikipedia Merkle tree page above (section on uses), and RFC 9162 (item 25) for a production design.
Check yourself. For 8 entries, how many sibling hashes does a proof contain, and why?

7. Domain separation and the odd-node rule

Definition. Domain separation: hashing leaves and inner nodes with different prefixes (here 0x00 and 0x01) so one cannot be mistaken for the other. Odd-node rule: an unpaired node is carried up unchanged rather than duplicated, because duplicating the last leaf makes two different logs share one root.
In this project. Fixed in `docs/proposal/decisions.md` and tested in `tests/test_vectors.py`.
Read. RFC 9162, Certificate Transparency Version 2.0 (its Merkle tree section; the older RFC 6962 is obsolete and was replaced by this one)
Check yourself. What attack does the 0x00/0x01 prefix prevent?

8. Tamper-evident versus tamper-proof

Definition. Tamper-proof means changes are impossible. Tamper-evident means changes cannot be stopped but will be detected. This project is tamper-evident, and only after the relevant batch has been anchored.
Read. Efficient Data Structures for Tamper-Evident Logging (Crosby and Wallach), the base paper, 17 pages.
Check yourself. What is protected, and what is not, between the last anchor and now?

B. Blockchain and smart contracts

9. Blockchain

Definition. A shared, append-only record kept by many computers, grouped into blocks that each include the hash of the previous block, so past data cannot be rewritten without being noticed.
In this project. An external witness that the log owner cannot control.
Read. Bitcoin: A Peer-to-Peer Electronic Cash System (Nakamoto, 2008), 9 pages, short and readable; Intro duction to Ethereum (ethereum.org)
Check yourself. Why can't the owner of the agent quietly change a root after it is written?

10. Ethereum accounts and transactions

Definition. An account has an address and is either controlled by a private key or by contract code. A transaction is a signed request from an account that changes the blockchain's state, such as calling a function.
In this project. The agent's account sends `anchor()` transactions; batches are stored per account.
Read. Ethereum accounts, Transactions
Check yourself. Why does the contract store batches per sender address?

11. Smart contract

Definition. A program stored on a blockchain at its own address. Its code and data are public and cannot be altered after deployment; callers can only use the functions it exposes.
In this project. `contracts/AuditAnchor.sol`.
Read. Introduction to smart contracts (ethereum.org), Introduction to Smart Contracts (Solidity docs)
Check yourself. What can and cannot a caller do to a batch after it is stored?

12. Ethereum Virtual Machine (EVM)

Definition. The runtime that every Ethereum node uses to execute contract code, metering each operation with gas.
Read. Ethereum Virtual Machine (ethereum.org)
Check yourself. Why must a contract's hashing give exactly the same answer on every node?

13. Gas

Definition. The unit that measures the computational work of an operation. Fees are gas used times the gas price. Writing storage is expensive; reading through a call is free.
In this project. Why only 32-byte roots go on-chain, and the subject of research question 2.
Read. Gas and fees (ethereum.org)
Check yourself. Why does anchoring a batch of 100 entries cost about the same gas as a batch of 5?

14. Solidity

Definition. The main programming language for Ethereum contracts, compiled to EVM bytecode plus an ABI (a description of the callable functions).
In this project. Version 0.8.28.
Read. Solidity by Example, and try code in Remix IDE (a browser editor, nothing to install).
Check yourself. Read `AuditAnchor.sol` line by line and explain each `require`.

15. Local development chain and Hardhat

Definition. A blockchain running on your own computer with fake money, used to build and test contracts quickly and for free. Hardhat is the toolkit that compiles contracts, runs that chain and runs tests.
In this project. `hardhat.config.js`, `npx hardhat node`, `npx hardhat test`. The project uses Hardhat 2, whose docs now live under v2.
Read. Hardhat's tutorial for beginners
Check yourself. What differs between a local chain and a public testnet, and why is the local one enough for the first evaluation?

16. ethers.js, Mocha and Chai

Definition. ethers talks to a blockchain from JavaScript; Mocha is the test runner; Chai provides the `expect(...)` assertions.
In this project. `chain-tests/AuditAnchor.test.js`.
Read. ethers.js v6 documentation, Mocha
Check yourself. Which line of the test checks that Python's proof verifies in Solidity?

17. web3.py

Definition. A Python library that talks to a blockchain node, to deploy contracts, send transactions and read stored data.
In this project. `src/app/chain_client.py`.
Read. web3.py documentation
Check yourself. How does `anchor_pending` know which batches are already anchored?

C. LLM agents and models

18. Large language model (LLM)

Definition. A neural network trained on large amounts of text that predicts a sensible continuation of a conversation.
Read. Building effective agents (Anthropic), a practical overview of how LLMs are used inside agents.
Check yourself. Why does the audit layer not depend on which model is used?

19. Tool calling and the agent loop

Definition. The model is told which functions it may use; it replies either with a final answer or with a request to call a tool. The program runs the tool, feeds the result back, and repeats until a final answer or a step limit.
In this project. `run_agent` in `src/domain/agent.py`; every step is logged.
Read. Ollama tool calling, and the same Anthropic article as item 18.
Check yourself. List the log entries produced by one tool call.

20. ReAct (reasoning and acting)

Definition. A pattern in which a model alternates between reasoning and taking actions that consult external tools, which reduces made-up answers.
Read. ReAct: Synergizing Reasoning and Acting in Language Models (Yao et al.)
Check yourself. Which parts of a ReAct trace does this project record?

21. Local models: Ollama and qwen3:8b

Definition. Ollama runs open-weight models on your own computer and serves them through a local web API. qwen3:8b is an 8-billion-parameter model (about 5 GB) that supports tool calling.
In this project. `src/app/ollama_llm.py`; runs need no API key and send nothing off the machine.
Read. Ollama on GitHub
Check yourself. Why are temperature 0 and a fixed seed set, and why can runs still differ slightly?

D. Python and testing

22. Immutable data and pure functions

Definition. A pure function depends only on its inputs and has no side effects; immutable data is never modified after creation, so you build new values instead.
In this project. Everything in `src/domain` and `src/core`; the log is a tuple passed along and returned. Python's `@dataclass(frozen=True)` makes the entry type immutable.

Read. dataclasses (Python docs)
Check yourself. Why does this style make the cryptographic code easier to test?

23. Automated testing with pytest

Definition. Writing small functions that assert what must be true, run automatically. Parametrised tests run one test over many inputs; fuzz (randomised) tests try many generated cases.
In this project. 79 Python tests; see `docs/TESTING.md`.
Read. Get started with pytest, How to parametrize fixtures and test functions
Check yourself. Why do the fuzz tests use fixed random seeds?

23a. JSON Lines, Git and Streamlit

JSON Lines. One JSON value per line; the log file format. jsonlines.org
Git. Version control for the project history. Pro Git (free book)
Streamlit. Turns a Python script into a web page; planned for the dashboard. Get started with Streamlit

---

E. Security thinking

24. Threat model

Definition. A statement of who might attack, what they can do, and what is out of scope. Here: an attacker with access to the stored log can edit it and recompute hashes, but cannot change a root already on the blockchain.
In this project. `docs/methodology/evaluation-plan.md` section 2.
Read. The Crosby and Wallach paper (item 8) defines an untrusted logger with auditors, the model closest to this project.
Check yourself. Name two attacks the system does not defend against.

25. Transparency logs (Certificate Transparency)

Definition. Public append-only Merkle-tree logs whose signed roots let anyone audit what was logged. A real, widely deployed version of the same idea.
Read. RFC 9162, Certificate Transparency Version 2.0 (long; read the introduction and the Merkle tree sections)
Check yourself. How does anchoring on a blockchain differ from signing tree heads?

26. Integrity versus truth

Definition. The blockchain proves a record was not changed after it was written. It cannot prove the record was true when written. Every claim in the thesis should respect this.
Check yourself. If the logger writes a false entry before anchoring, what does the verifier report?

F. Research and writing

27. AI accountability and governance

Definition. Keeping records and controls so that the behaviour of AI systems can be traced and reviewed; the motivation for audit trails.
Read. AI Risk Management Framework (NIST)
Check yourself. Which function of the framework does an audit trail support?

28. Ethereum protocol reference

Definition. The formal specification of Ethereum, for precise statements about gas and the EVM. Advanced; use as a reference, not as a first read.
Read. Ethereum Yellow Paper (Wood)

29. Reading a research paper efficiently

Read in this order: abstract, introduction, conclusion, figures, then the method. For each paper note the problem, the idea, the evidence, and what it leaves open. Do this first for Crosby and Wallach (items 8 and 24).

Master reading list (all verified links)

#	Resource	Type
1	Cryptographic hash function	Wikipedia
2	FIPS 180-4 Secure Hash Standard	Standard
3	Python hashlib	Docs
4	Merkle tree	Wikipedia
5	Crosby and Wallach, tamper-evident logging	Paper (base paper)
6	RFC 9162 Certificate Transparency 2.0	Standard
7	Bitcoin whitepaper	Paper
8	Introduction to Ethereum	Docs
9	Ethereum accounts	Docs
10	Transactions	Docs
11	Smart contracts	Docs
12	EVM	Docs
13	Gas and fees	Docs
14	Solidity: introduction to smart contracts	Docs
15	Solidity by Example	Tutorial
16	Remix IDE	Online tool

#	Resource	Type
17	Hardhat tutorial (v2)	Tutorial
18	ethers.js v6	Docs
19	Mocha	Docs
20	web3.py	Docs
21	Building effective agents (Anthropic)	Article
22	Ollama tool calling	Docs
23	Ollama on GitHub	Project
24	ReAct paper	Paper
25	Python dataclasses	Docs
26	Python json	Docs
27	pytest: get started	Tutorial
28	pytest: parametrize	Docs
29	JSON Lines	Spec
30	Pro Git	Book
31	Streamlit: get started	Tutorial
32	NIST AI Risk Management Framework	Framework
33	Ethereum Yellow Paper	Spec

Only the Python json page was not opened individually; the other Python pages were. If it ever fails, search "Python json module".